# Ders 18 Genişletilmiş Notu: Tip Dönüşümü, Dosyalarla Çalışma ve Serileştirme, İç Sınıflar ve GUI'ye Giriş

## Ders 18 Genişletilmiş Notu: Tip Dönüşümü, Dosyalarla Çalışma ve Serileştirme, İç Sınıflar ve GUI'ye Giriş

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

## İçindekiler


- 1. Nesneler Arasında Tip Dönüşümü
- 2. Market Rafı Örneği
- 3. Dosyalarla Çalışmanın Temelleri
- 4. Dosya Yönetimi, I/O İşlemleri ve File Sınıfı
- 5. Akışlar ve Serileştirme
- 6. Telefon Rehberi Uygulaması
- 7. Dosya Sonu Sorunu ve Kapsayıcı Nesneler
- 8. İç Sınıflar
- 9. Görsel Programlamaya Giriş: C# ile Pencere ve Olaylar
- 10. Ders Sonu Sorusu: İç Sınıfların Bellek Maliyeti ve Java'da Olay Bağlama


## 1. Nesneler Arasında Tip Dönüşümü

Hoca derse, yaklaşık iki haftadır ders yapılamadığını, bu yüzden biraz hızlanacağını söyleyerek başladı. Hedefi thread konusuna kadar gelmekti.

### 1.1 İlkel Tiplerden Sınıflara

Hoca önce ilkel tiplerdeki dönüşümü hatırlattı. `double` bir `d` değişkenine 3.2 verip sonra `int i = d` yazdığımızda arada gizli bir dönüşüm olduğunu, buna örtük (implicit) dönüşüm dendiğini, dönüşümü kendimiz yazarsak açık (explicit) dönüşüm olduğunu söyledi. Tipler arası dönüşümde dikkat edilecek şey boyut, yani detay bilgisinin kaybolup kaybolmamasıdır.

> **Ders sonrası not:** Hoca `double d = 3.2; int i = d;` satırında örtük dönüşüm olduğunu söyledi; bu satır derlenmez. javac ile denendi: `incompatible types: possible lossy conversion from double to int`. Doğrusu `int i = (int) d;` biçiminde açık dönüşümdür. Örtük dönüşüm bilgi kaybı olmayan yönde olur: `int k = 5; double e = k;` javac ile denendi ve derlendi.

Fakat bu dersteki tip dönüşümü (typecasting) ilkel tiplerle ilgili değildir. Kastedilen, tasarladığımız herhangi bir sınıf üzerinde yapılan dönüşümdür. Java terminolojisi bu kelimeyi nesnenin türünün dönüştürülmesi için kullanır.

### 1.2 Yukarı ve Aşağı Dönüşüm

Hoca kalıtım kuralını hatırlattı. `A` diye bir sınıf tasarlayıp `B`'yi ondan türettiysek `A a = new B();` yazabiliyorduk. Referans üst sınıf türünden, nesnenin kendisi alt sınıf türündendir. Bunun tam

tersi doğrudan mümkün değildir: `B b = new A();` javac ile denendi, `incompatible types: A cannot be converted to B` hatası verir.

Özel bir nesneyi daha genel bir nesneye kayıpsız çevirebiliriz. Hocanın örneği şuydu: Bir kullanıcı sınıfı ve ondan türeyen öğrenci ve öğretmen sınıfları olsun. Her öğretmen aynı zamanda bir kullanıcıdır, dolayısıyla öğretmen nesnesini kullanıcı türünden yazabilirsiniz. Buna tip güvenli işlem (type safe operation) denir ve otomatik yapılır.

Tersi, yani üst sınıftan alt sınıfa dönüşüm risklidir. Elle yapılması gerekir. Doğrudan yapılırsa farklı istisnalara, programın kapanmasına ya da yanlış sonuçlara yol açabilir. Hoca kuralların Java'nın yeni sürümlerinde sürekli değiştiğini ve kolaylaştırıldığını, ama temelini bilmekte fayda olduğunu ekledi.

### 1.3 Manuel Dönüşümün Kuralları

Hoca, "dönüşüm" sözcüğünün geniş kullanıldığını, örneğin toplama operatörüyle `String` ile `int` toplanıp `String` elde edildiğini hatırlattı. Nesneler üzerinde ise elle dönüşüm iki durumda yapılır:

- Arayüz (interface) referansından sınıfa.
- Üst sınıftan (superclass) alt sınıfa (subclass).

İkisi de güvensiz işlemdir, bu yüzden öncesinde mutlaka bir kontrol mekanizması kurulur. javac ile denendi: `A a2 = new A(); B b2 = (B) a2;` derlenir ama çalışırken `ClassCastException` verir. Referansın gerçekten `B` nesnesini gösterdiği durumda (`A a = new B(); B b = (B) a;`) dönüşüm sorunsuz çalışır. Yani derleyici dönüşümü kabul etse de doğruluğu çalışma anında belirlenir.

## 2. Market Rafı Örneği

### 2.1 Item Arayüzü, Food ve Electronic

Örnekte `Item` türünde bir arayüz var. Bu ürünlerden biri gıda (`Food`), biri elektronik ürün (`Electronic`). Her birinin kendine özgü özellikleri olabilir. Örneğin son kullanma tarihi gıdada vardır, elektronik üründe yoktur. Market rafı (`Shelf`) bu ürünlerin listesini tutar.

Burada arayüzün önemli bir özelliği kullanılıyor: Arayüzden nesne üretilemez ama arayüz türünden referans alınabilir. Bu sayede gıdaların, elektroniklerin ve diğer ürünlerin referansları aynı `LinkedList<Item>` içinde tutulabilir. Hoca bunun soyut sınıf yerine arayüz kullanılarak yapıldığına dikkat çekti.

### 2.2 Shelf Sınıfı

Bağlı liste yapıcı metotta oluşturulur. Bir ürünü aramak için bütün kayıtlara bakılır. Bulunursa `true`, bulunamazsa `false` döner. Ürün eklerken önce daha önce eklenip eklenmediğine bakılır, eklenmemişse listeye eklenir. Hoca bunların basit işlemler olduğunu, asıl önemli metodun `printExpiredItems` olduğunu söyledi.

### 2.3 instanceof ve Çift Parantezli Dönüşüm

`printExpiredItems` metodunda önce `hasExpiredItem` adlı bir bayrak değişkeni vardır. Foreach döngüsüyle listedeki her ürün sırayla alınır. Ürün elektronik de olabilir gıda da, ama son kullanma özelliği yalnızca gıdada vardır.

Önce `item instanceof Food` ile nesnenin `Food` türünden üretilip üretilmediği sorulur. Sonuç `true` ya da `false` döner. Burada referans `Item` türünde, nesnenin kendisi `Food` türündedir. `Item` hiyerarşide üstte, `Food` altta olduğu için test doğru çıkar.

`isExpired` metoduna ulaşmak için ise `Item` referansını `Food`'a çevirmek gerekir. Hocanın dikkat çektiği yazım şudur:

```
if (item instanceof Food)
    if (((Food) item).isExpired()) {
        hasExpiredItem = true;
```

```
            System.out.println(item);
        }
```

Yani önce nesnenin önüne dönüşüm yazılır, sonra bütün ifade yeniden paranteze alınır. Dönen referans üzerinden `Food` sınıfındaki metoda erişilir. Sonuç `true` ise ürün yazdırılır, değilse döngü devam eder.

> **Ders sonrası not:** Hoca `instanceof` için "metot" dedi ve dış parantezi "tekrar cast işlemi" gibi anlattı. `instanceof` bir operatördür. Dönüşüm bir kez yapılır, dış parantez yalnızca gruplama içindir: `(Food) item.isExpired()` yazılırsa `.` önce işlenir ve derleyici `isExpired`'ı `Item` içinde arar. javac ile denendi: `cannot find symbol` hatası verir.

> **Ek bilgi (derste anlatılmadı):** Hocanın sözünü ettiği yeni Java kolaylıklarından biri, `instanceof` ile dönüşümü tek adımda birleştiren yazımdır: `if (o instanceof String s) System.out.println(s.length());`. Bu satır JDK 25'te javac ile denendi ve derlendi. Burada `s`, kontrol doğru çıkarsa kullanılabilen, dönüştürülmüş bir değişkendir. Ayrı bir `(String) o` yazmaya gerek kalmaz.

### 2.4 Calendar Sınıfı ve Fabrika Metodu

Son kullanma tarihini denemek için Java'daki takvim sınıfı `Calendar` kullanılmış. Hoca bu sınıfla birçok takvim çeşidinin ayarlanabildiğini söyledi. Kodda `Calendar cal = Calendar.getInstance();` yazılıyor.

**Soru:** `getInstance` nasıl tanımlanmış bir metottur? **Cevap:** Statik bir metottur ve geriye bir `Calendar` nesnesi döndürür.

Nesneyi `new` ile değil, sınıfın statik bir metoduyla üretmeye fabrika metodu (factory method) denir. Hoca metodun içinde sistemin saat bilgisinin de alındığını tahmin ettiğini, ayrıntısını şu an bilmediğimizi belirtti. Sonra `cal.add(...)` ile tarih üzerinde gün gibi birimlerde değişiklik yapılır, `cal.set(...)` ile yıl, ay, gün ayarlanır, `cal.getTime()` ile tarih bilgisi alınır.

> **Ders sonrası not:** Hoca "normalde `Calendar cal = new Calendar()` diyebilirdik" ve "statik metodun içinde `new Calendar` var" dedi. `Calendar` soyut bir sınıftır. javac ile denendi: `Calendar is abstract; cannot be instantiated`. `Calendar.getInstance()` çalıştırılıp türü yazdırıldığında `java.util.GregorianCalendar` çıktı, yani dönen nesne alt sınıftandır.

> **Ders sonrası not:** Hoca `cal.set` ile bilgilerin "bilgisayarınıza set edildiğini" söyledi. Çalıştırılarak denendi: `c.set(2000, 0, 1)` yalnızca `Calendar` nesnesini değiştirdi, `new Date()` güncel tarihi vermeye devam etti. Sistem saati değişmez. Ayrıca `getTime()` yalnız saat değil, tam tarih içeren bir `Date` döndürür.

### 2.5 Food Sınıfı ve Tarih Karşılaştırması

`Food` sınıfı `Item` arayüzünü uygular. Son kullanma tarihini `Date` nesnesi olarak alır ve saklar. `isExpired` metodunda önce içinde bulunduğumuz gün elde edilir, sonra `expireDate.before(today)` çağrılır. Son kullanma tarihi bugünden önceyse `true` döner, yani ürünün tarihi geçmiştir.

Hoca bu tür tarih kontrollerini günlük hayata bağladı. Veri tabanlarında da benzeri yapılır. Doğum gününüzde saat 12'yi geçince bazı kurumlardan gelen mesaj, günlük çalışan bir tetikleyicinin (trigger) sonucudur. Burada aynı şeyi programlama tarafında yapıyoruz.

> **Ders sonrası not:** Hoca `before` için "günden günü, aydan ayı, yıldan yılı çıkarmaya çalışıyor" dedi. Karşılaştırma milisaniye düzeyindedir. Çalıştırılarak denendi: `new Date(1000).before(new Date(1001))` sonucu `true`. Dolayısıyla son kullanma tarihi bugünün daha erken bir saatiyse ürün de "geçmiş" sayılır.

### 2.6 Bütün Kod

Hoca kodu slayttan anlattı. Aşağıdaki sürüm bu anlatımdan kurulmuştur. Derlenip çalıştırıldı (javac, JDK 25).

```java
// Item.java
public interface Item {
    String getName();
}

// Food.java
import java.util.Date;

public class Food implements Item {
    private String name;
    private Date expireDate;

    public Food(String name, Date expireDate) {
        this.name = name;
        this.expireDate = expireDate;
    }

    public String getName() { return name; }
    public Date getExpireDate() { return expireDate; }

    public boolean isExpired() {
        Date today = new Date();
        return expireDate.before(today);
    }

    public String toString() { return name; }
}

// Electronic.java
public class Electronic implements Item {
    private String name;

    public Electronic(String name) { this.name = name; }

    public String getName() { return name; }

    public String toString() { return name; }
}

// Shelf.java
import java.util.LinkedList;

public class Shelf {
    private LinkedList<Item> items;

    public Shelf() { items = new LinkedList<Item>(); }

    public boolean findItem(Item aranan) {
        for (Item item : items)
            if (item.getName().equals(aranan.getName()))
                return true;
        return false;
    }

    public void addItem(Item item) {
        if (!findItem(item))
            items.add(item);
    }

    public void printExpiredItems() {
```

```
        boolean hasExpiredItem = false;
        System.out.println("Expired item(s): ");
        for (Item item : items) {
            if (item instanceof Food)
                if (((Food) item).isExpired()) {
                    hasExpiredItem = true;
                    System.out.println(item);
                }
        }
        if (hasExpiredItem == false)
            System.out.println("All items are fresh!");
    }
}

// Market.java
import java.util.Calendar;
import java.util.Date;

public class Market {
    public static void main(String[] args) {
        Calendar cal = Calendar.getInstance();
        cal.add(Calendar.DAY_OF_MONTH, -1);
        Date past = cal.getTime();

        cal = Calendar.getInstance();
        cal.add(Calendar.DAY_OF_MONTH, 1);
        Date future = cal.getTime();

        Shelf shelf = new Shelf();
        shelf.addItem(new Food("Eski gida", past));
        shelf.addItem(new Food("Yeni gida", future));
        shelf.addItem(new Electronic("Elektronik urun"));
        shelf.printExpiredItems();
    }
}
```

Çıktı:

```
Expired item(s):
Eski gida
```

Akış şöyledir:

1. `main` geçmiş ve gelecek tarihli iki gıda ile bir elektronik ürünü rafa ekler.
2. `printExpiredItems` üç ürünü sırayla dolaşır.
3. Elektronik ürün `instanceof Food` testini geçemez ve atlanır.
4. İki gıdadan yalnız geçmiş tarihli olanın `isExpired` sonucu `true` olur ve yazdırılır.

### 2.7 Tip Dönüşümünü Az Kullanın

Slaytta tip dönüşümü için "necessary, but use sparingly" benzeri bir ifade vardı. Hoca bunu "gerekli yerlerde kullanın ama çok da kullanmayın" diye açıkladı. Java'nın ilk sürümlerinde bu daha gerekliydi. Derleyiciler güncellendikçe eklenen özellikler ihtiyacı azalttı. Örneğin bu işi soyut sınıflarla da yapabiliriz. Hocaya göre tip dönüşümü asıl olarak ters serileştirmede, yani dosya ve giriş-çıkış işlemlerinde önemlidir. Bu derste paylaşılan kod klasöründe aynı örneğin soyut sınıflarla, tip dönüşümsüz bir sürümü de vardır.

> **Hocanın vurgusu:** Bir sınavda size bir UML şeması verilirse ve `B`'den `A`'ya, `C`'den `A`'ya dönüşüm varsa sorun yoktur. `A`'dan `B`'ye tarzında bir dönüşüm gerekiyorsa buradaki mantıkla, yani kontrol edip elle dönüştürerek yapmanız gerekir.

Hoca, konunun biraz eskidiğini de söyledi. İnternette "Java typecasting" diye arayınca daha çok kutulama (boxing, unboxing), `parse` metotları ya da değerin önüne `int` yazma çıkıyor. Ama

nesneler üzerinde de bu mümkündür.

## 3. Dosyalarla Çalışmanın Temelleri

Dosyalarla çalışma, yapısal programlamada C ile görülmüştü. Bu dönem onun üzerine ekleme yapılacak.

### 3.1 Dosyalarla Çalışmak Ne Demek?

Dosyalarla çalışmaktan kasıt şunlardır:

- Dosyanın oluşturulması, içeriğinin değiştirilmesi, silinmesi.
- Öz niteliklerinin değiştirilmesi: ad değiştirme, kopyalama.
- Klasör oluşturma ve dosyaları düzenleme. MS-DOS ekranında ya da Linux kabuğunda `cd`, `md` ile yaptığımız işler.

Bunların programla yapılması bazen zorunludur. Hocanın örneği görüntü işleme ödeviydi. Adları `img01`, `img02` diye giden bir milyon PNG dosyası olsun. Bunları programa tek tek vermek mümkün değildir. Döngüyle dosya adları `String` olarak üretilir, o adlarla dosyalar açılır, işlenir ve yenileri kaydedilir.

### 3.2 Sıralı ve Rastgele Erişim

Dosyanın içeriğini okumanın iki yolu vardır:

- **Sıralı erişim (sequential access):** Dosyanın başından başlar, sonuna kadar bütün bilgileri tek tek okur. Hoca bunu eski teyplere benzetti: İstediğiniz noktaya gitmek için baştan ileri sararsınız, fazla gittiyseniz geri gelirsiniz.
- **Rastgele erişim (random access):** Dosyada istediğiniz konuma doğrudan gidip orayı okursunuz.

İkisinde de bir imleç (cursor) kullanılır. Dosya açılınca imleç baştadır, metin editöründe yanıp sönen imleç gibi düşünebilirsiniz. Bir karakter, bir satır, bir `String` ya da bir veri bloğu okumak isteyebilirsiniz. Okuma yapılınca imleç, okuduğunuz birim kadar ilerler ve sonraki okuma bir sonraki birimden devam eder. Hoca özellikle "birim" dedi, çünkü bu her zaman karakter değildir.

### 3.3 Nasıl Yazıldıysa Öyle Okunur

Neden bit, bayt, karakter ve paragraf gibi farklı okuma birimleri var? Çünkü dosyalar nasıl okunmak isteniyorsa ona göre yazılır, yani o şekilde düzenlenir.

> **Hocanın vurgusu:** Dosyayı nasıl yazıldıysa ona göre okumamız lazım. Bunun için dosyanın nasıl yazıldığını, yani formatını (data sheet) bilmemiz gerekir. Özellikle ikili dosyalarda her bitin anlamı vardır. Bitlerden birinin yeri değişirse dosya da değişir.

### 3.4 PDF Deneyi

Hoca bunu bir deneyle gösterdi:

1. Bir PDF dosyasını kopyaladı ve kopyasını metin editöründe açtı.
2. Bir kısmını silip kaydetti, PDF'i yeniden açtı. Dosya yine açıldı. Demek ki hayati bir kısım silinmemişti, silinen veriydi.
3. Daha fazla bozmayı denedi. Dosyanın `stream`, `endstream`, `trailer` gibi yapıları görünüyordu. Hoca, PDF okuyucuların bu formattaki verileri gerektiğinde bit bit, gerektiğinde bayt bayt ya da satır satır okuyup içeriği gösterdiğini söyledi. Bazı yerler bozuk görünmeye başladı.
4. Dosyanın öz niteliklerini tutan bir yeri bozunca okuyucu dosyanın bozuk (corrupt) olduğunu söyledi ve açmadı.

Sonuç şudur: Veri kısmı bozulursa dosya kısmen açılabilir. Başlık (header) gibi format bilgisi bozulursa okuyucu dosyayı anlayamaz.

### 3.5 İkili ve Metin Format

İki tür dosya formatı vardır: ikili (binary) ve metin (text). İkili formatta veriler kodlanmış olarak saklanır. Hoca bunun nedenini sayısal bir örnekle gösterdi.

**Verilen:** 8 bitlik bir dizi, `01010001`. Hoca bunu `.txt` dosyasına karakter karakter yazdı.

**Metin olarak boyut:** Dizinin her karakteri bir bayt yer kaplar, dosya 8 bayt olur. Uzantıyı `.c` ya da başka bir şey yapmak bunu değiştirmez, yine 8 bayt yazılır. Oysa bu aslında 8 bitlik, yani 1 baytlık bir veridir. python ile denendi: Aynı dizi metin olarak 8 bayt, değeri tek bayt olarak yazılınca 1 bayt tuttu.

**Sayısal değer:** Hoca birler ve on altılar basamağından başlayarak topladı:

$$1 \cdot 2^0 + 1 \cdot 2^4 + 1 \cdot 2^6 = 1 + 16 + 64 = 81$$

Ara toplamlar $1 + 16 = 17$ ve $17 + 64 = 81$'dir (python ile hesaplandı).

**Karakter karşılığı:** Hoca 65'in büyük `A` olduğunu hatırlatıp 81'in karşılığını yaklaşık olarak aradı ve "büyük M" dedi.

> **Ders sonrası not:** Hoca 81'in harf karşılığı için "M" dedi. python ile hesaplandı: 81 = `Q`, 77 = `M`, 76 = `L`, 85 = `U`.

Hocanın vardığı nokta şudur: Bu değer ASCII ya da Unicode olarak açıldığında ekranda tek bir karakter görürüz. Sonra program onu yeniden 0 ve 1'lere çevirir. O bitlerin ne anlama geldiği, yani rakam mı, görüntüdeki renk mi olduğu, formata göre belirlenir.

Hoca ikili formatın amacı olarak yer kazanmayı ve daha güvenli hale getirmeyi saydı. İkili veri hızlı ve güvenlidir ama insan tarafından okunamaz, metin formatında ise durum tersidir.

> **Ders sonrası not:** İkili format tek başına güvenlik sağlamaz. Bu dersin 6.6 numaralı kısmında görüleceği gibi, `contacts.dat` ikili olduğu halde içine yazılan adlar metin editöründe okunabiliyor.

### 3.6 Dosya İşlemlerinde İstisnalar

Hoca, Java'da dosyalarla çalışırken C'dekine göre daha iyi koruma mekanizmaları olduğunu söyledi. Karşılaşılabilecek istisnalar şunlardır:

- **`IOException`:** Giriş-çıkış işlemi sırasında oluşan hataların genel türü.
- **`EOFException`:** `IOException`'dan türer. Dosyanın sonuna beklenmedik biçimde ulaşılırsa fırlatılır.
- **`FileNotFoundException`:** `IOException`'dan türer. Dosya bulunamadığında fırlatılır.
- **`SecurityException`:** `RuntimeException`'dan türer, güvenlikle ilgilidir. Hoca bunu dosyayı açma yetkinizin olmadığı durumla örnekledi.

Hiyerarşi çalıştırılarak doğrulandı (`isAssignableFrom`). İki alt tür `IOException`'dan türediği için yalnız `catch (IOException e)` yazmak hepsini yakalar. Hocanın dediği gibi genellikle bu yeterlidir.

> **Hocanın vurgusu:** Bu istisnalara hâkim olmak gerekir. Dosyalarla çalışırken `try-catch` bloklarını kullanmak lazım. Böylece hata kullanıcıya bildirilir ve programın kapanması engellenir.
>
> **Ders sonrası not:** Hoca "`catch`'in birincisine `IOException`, sonra `EOFException`, sonra `FileNotFoundException` yazabilirim" dedi. Bu sıra derlenmez. javac ile denendi: `exception EOFException has already been caught`. Ayrıntılı `catch` yazılacaksa önce alt sınıflar, en son `IOException` gelir; bu sıra javac ile denendi ve derlendi.
>
> **Ders sonrası not:** Hoca dosya silme girişiminde de `FileNotFoundException` alınacağını söyledi. Çalıştırılarak denendi: Olmayan bir dosya için `new File(...).delete()` istisna fırlatmadı, `false` döndürdü.

> **Ders sonrası not:** Hoca işletim sisteminin izin vermediği durumu `SecurityException` ile anlattı. Linux'ta okuma izni kapatılan dosya `FileInputStream` ile açılınca `java.io.FileNotFoundException: ... (Permission denied)` alındı (çalıştırılarak denendi). `SecurityException` büyük olasılıkla işletim sisteminin değil, Java'nın kendi güvenlik denetiminin istisnasıdır.

## 4. Dosya Yönetimi, I/O İşlemleri ve File Sınıfı

### 4.1 İki Ayrı Alan

Hoca dosya işlemlerini ikiye ayırdı:

- **Dosya yönetimi:** Dosya oluşturma, adını değiştirme, silme, klasör oluşturma, klasörleri hiyerarşik düzenleme.
- **Giriş-çıkış (I/O) işlemleri:** Veri yazma ve okuma.

I/O yalnızca dosyayla yapılmaz. Klavyeden sayı girmek de bir I/O işlemidir. TCP soketleri de öyledir, bunlar ağ derslerinde görülecek. Hocanın örneği WhatsApp'tı: Arkadaşınıza gönderdiğiniz mesaj telefonunun IP numarasına gider, sonra soket numaraları aracılığıyla kurulan TCP bağlantısıyla telefondaki uygulamaya düşer. Uygulama da gelen mesajı açıp gösterir. Hoca bunun, programın işletim sistemiyle "kol kola" çalıştığı bir haberleşme olduğunu söyledi.

> **Ders sonrası not:** Örnek basitleştirilmiştir. Hoca mesajın doğrudan arkadaşın telefonunun IP numarasına gittiğini söyledi; bir mesajlaşma uygulamasında mesaj büyük olasılıkla önce uygulamanın sunucusuna gider.

Bu kaynakların hepsinde temel fark, gelen verinin formatıdır. Veri nasıl yazıldıysa öyle okunur: TCP soketinin bir paket yapısı vardır. Web sayfalarının da bir yapısı vardır, tarayıcı HTML ve CSS etiketlerini çözerek sayfayı gösterir.

Slayttaki ifadeye göre buradaki yaklaşım nesneye yönelik programlamanın doğasıyla uyumludur. Hoca bunu şöyle açtı: Şimdiye kadar ilkel tipler değil, sınıflar ve nesneler üzerinde konuştuk. Dosya işlemlerini de bu kapsamda düşünmeliyiz. Yani dosyaya bir nesneyi ya da bir nesne listesini yazmak, bir `HashMap`'in tamamını tek seferde yazıp okumak. İçinde milyon veri olan bir bağlı listeyi tek tek gezip okumak ve yazmak çok anlamsızdır. Bunun yan etkisi olarak karmaşıklık artar.

### 4.2 File Nesnesi Dosya Oluşturmaz

Dosya yönetimi `java.io.File` sınıfıyla yapılır. Hoca önemli bir ayrım yaptı: Bir dosya nesnesi oluşturmak ile diskte dosya oluşturmak farklı şeylerdir.

```
File f = new File("deneme.txt");
```

Bu satır diskte dosya oluşturmaz. Dosyanın ayrıca var hale getirilmesi gerekir. Çalıştırılarak denendi: `new File("yok_dosya.txt").exists()` sonucu `false`.

> **Ders sonrası not:** Hoca "`new File` diyeceğiz, belki bunu boş bırakacağız" dedi. Parametresiz yapıcı yoktur. javac ile denendi: `no suitable constructor found for File(no arguments)`.

### 4.3 Mutlak ve Göreli Yol

Yapıcıya verilen ad bir yol içerebilir, dosya ya da klasör adı olabilir:

- **Mutlak yol (absolute path):** `C:` gibi bir kökten başlayıp hedefe kadar bütün klasörleri yazmak. Program o hedefte dosya oluşturur ya da oradaki dosyayı açmaya çalışır.
- **Göreli yol (relative path):** Programın çalıştığı klasöre göre yazılan yol. Yalnızca `deneme.txt` yazarsanız dosya, programın o an çalıştığı klasörde oluşur.

Hoca göreli yolların zorlayıcı olabileceğini, çünkü programlarımızı genellikle Eclipse ya da NetBeans'in çalışma klasörü altında tuttuğumuzu söyledi. Gerçek hayatta çok kullanılır. Grafik arayüzden hedef klasörü seçtiğiniz kaydetme pencerelerini de buna örnek verdi.

**Ders sonrası not:** Hoca `../YeniKlasor/deneme.txt` için "bulunduğum klasörün altındaki klasör" dedi. `..` bir üst klasördür. Çalıştırılarak denendi: `getCanonicalPath` ile `.../t1` klasöründe `../YeniKlasor/deneme.txt` yolu `.../YeniKlasor/deneme.txt`, `YeniKlasor/deneme.txt` yolu ise `.../t1/YeniKlasor/deneme.txt` verdi. Çalışma klasörünün altındaki klasör için `..` yazılmaz.

**Ders sonrası not:** Kaydetme pencereleri göreli yol örneği olarak verildi; bu pencereler büyük olasılıkla seçilen konumu mutlak yol olarak döndürür.

### 4.4 Yol Ayırıcıları

Windows ters eğik çizgi, Unix normal eğik çizgi kullanır. Java'da metin içinde ters eğik çizgi özel anlam taşıdığı için iki kez yazılır:

```
File f = new File("C:\\klasor\\deneme.txt");
```

Bu metin çalıştırılınca `C:\klasor\deneme.txt` olarak yazdırıldı. Uzantı yazmazsanız uzantısız bir dosya olur. Taşınabilirlik için `File.separator` ve `File.separatorChar` değişkenleri vardır. Linux'ta ikisi de `/` verdi (çalıştırılarak denendi).

### 4.5 Diğer Yapıcılar ve Faydalı Metotlar

Yol ve ad ayrı ayrı da verilebilir: `File(String yol, String ad)`. Var olan bir klasörün `File` nesnesi de verilebilir: `File(File ust, String ad)`.

Sınıfın faydalı metotları şunlardır. Hoca bunların ezberlenmesi gerekmediğini söyledi.

- `exists()`: Var mı?
- `isFile()`, `isDirectory()`: Dosya mı, klasör mü?
- `getParentFile()`: Hangi klasörün altında?
- `getCanonicalPath()`: Tam yolu verir. `IOException` fırlatabilir.
- `canRead()`, `canWrite()`: Okunabilir mi, yazılabilir mi?
- `createNewFile()`: Yeni dosya oluşturur.
- `mkdir()`: Yeni klasör oluşturur. `mkdirs()`: Alt alta 3-4 klasörü birlikte oluşturur.
- `renameTo()`, `delete()`: Ad değiştirir, siler.

**Ders sonrası not:** Hoca `getCanonicalPath` için "öyle bir dosya olmayabilir" gerekçesiyle `IOException` fırlatabileceğini söyledi. Çalıştırılarak denendi: Olmayan bir dosya ve klasör için de istisnasız tam yol döndü. Yalnızca bu nedenle istisna oluşmaz, ama metot `IOException` bildirdiği için yine de yakalanmalıdır.

## 5. Akışlar ve Serileştirme

### 5.1 Akış (Stream)

Java'da, C++'ta olduğu gibi, herhangi bir I/O kaynağı akış (stream) olarak ifade edilir. Programınız dış dünyayla haberleşmek zorundadır. En basiti `System.out.print` ya da `printf`'tir. Dosya, bellek, komut satırı, ağ, yazıcı, tarayıcı hep bu kapsamdadır.

Akışlar ikili ya da metin formatında olabilir. Java'nın güzel bir özelliği, farklı çalışma biçimlerindeki akışları birbirine zincirleyebilmesidir (chain). Bu derste amaç, nesneye yönelik terminolojiye uygun olarak bir akışa nesne yazmak ya da akıştan nesne okumaktır. Kitap, kalem gibi nesnelerimizi bu yolla yazıp okuruz.

### 5.2 Serileştirme

Nesnenin tamamını diske yazma işlemine serileştirme (serialization) denir. Bir çıkış işlemi gibi düşünebiliriz. Ters işlem, yani okuma ise ters serileştirmedir (deserialization).

Yazılacak nesnelerin sınıfı `Serializable` arayüzünü uygulamalıdır. Bu arayüz için ayrıca metot yazılmaz. Sınıfın yazılabilir olduğunu gösteren bir işaretçi, bir bayrak gibi çalışır (marker interface).

Serileştirmede `ObjectOutputStream` ve `FileOutputStream` birlikte kullanılır. `ObjectOutputStream` nesneler üzerinde çalışmak için geliştirilmiştir. Ters serileştirmede ise bunların `InputStream` karşılıkları vardır.

> **Ders sonrası not:** Hoca "önemli olan ilkel veri tiplerinin olmaması, `int` yerine sarmalayıcısıyla gönderin" dedi. Bu kesin bir kural değildir. Çalıştırılarak denendi: `int` alan içeren sınıf sorunsuz yazılıp okundu. `writeObject(5)` çağrısı okunurken `java.lang.Integer` olarak geldi, yani değer otomatik kutulandı. İlkel değerler için `writeInt` ve `readInt` de vardır, bunlar da denendi.

### 5.3 transient

Hoca, bir sınıfın alanlarından birinin serileştirilmesini istemiyorsak ona `transient` anahtar kelimesini eklediğimizi söyledi. Bu kısmı kısa geçti ve kod göstermedi. Çalıştırılarak denendi: `transient String` alan yazılıp okununca `null` geldi, normal alanlar korundu.

## 6. Telefon Rehberi Uygulaması

Basit bir rehber uygulaması: Kişilerin adını, telefonunu ve e-posta adresini saklayıp dosyaya aktaracağız. Kodlar `nyp12a`, `nyp12b`, `nyp12c` paketlerindedir. Çalıştırılan dosyanın içeriğinden paket adının `files` olduğu görülüyor.

### 6.1 Contact Sınıfı

Kişi sınıfı `Serializable` arayüzünü uygular. Adla çağrılan bir yapıcısı, `get` ve `set` metotları ve yazdırma için `toString` metodu vardır. Hoca, buraya kadar hiçbir yazma işlemi yapılmadığına dikkat çekti.

```
package files;

public class Contact implements java.io.Serializable {
    private static final long serialVersionUID = 1L;
    private String name, phone, eMail;

    public Contact(String name) { this.name = name; }

    public String getName() { return name; }
    public String getPhone() { return phone; }
    public String getEMail() { return eMail; }
    public void setPhone(String phone) { this.phone = phone; }
    public void setEMail(String eMail) { this.eMail = eMail; }

    public String toString() {
        return name + " - " + phone + " - " + eMail;
    }
}
```

### 6.2 serialVersionUID

Slaytta altı çizili ikinci şey `serialVersionUID`'dir. Hoca bunu sürüm bilgisi taşıyan bir değişken olarak anlattı. Programınız ve dosyalarınız güncellenebilir, birden çok sürümde okuma yazma yapılabilir. Nesneyi yazdığınızda bu değer de akışa gönderilir. Yazmak zorunda değiliz, ama yazmazsak uyarı alırız. javac ile denendi: `-Xlint:serial` ile `serializable class X has no definition of serialVersionUID` uyarısı geldi.

> **Ders sonrası not:** Hoca değişkeni "okuma yazma yapıldığında versiyon numarasını başlangıç değerinden başlayarak yazan" bir şey gibi anlattı. Her yazmada artan bir sayaç değildir, sınıfın sürüm kimliğidir ve elle verilir. Çalıştırılarak denendi: Değeri `1L` iken yazılan nesne, sınıf `2L` ile yeniden derlenip okununca `java.io.InvalidClassException: ... stream classdesc serialVersionUID = 1, local class serialVersionUID = 2` verdi.

### 6.3 Annotation

Uyarıları gizlemek için satır başında `@` işaretiyle yazılan annotation kullanılır, örneğin `@SuppressWarnings`. Annotation'lar bilgi hakkında bilgi, yani üst veri (metadata) taşır. IDE'ye, derleyiciye ya da başka bir programa ek bilgi verir. En basit örnek, metodu geçersiz kıldığımızı söyleyen `@Override`'dır. Hoca uyarıların aslında dikkate alınması gerektiğini de ekledi.

### 6.4 CreateContacts: Önce Sayı, Sonra Kayıtlar

Program kullanıcıya kaç kişi gireceğini sorar ve bu sayıyı `Integer` olarak saklar. Bunun nedeni önemlidir: Dosyada kaç kayıt olduğunu saklamazsak okurken dosyayı baştan sona bir kez geçip saymamız gerekir. Özellikle dizi gibi dinamik olmayan bir yapı kullanıyorsak bu bilgiye kesinlikle ihtiyaç vardır.

Sonra o boyutta bir `Contact` dizisi açılır ve kullanıcıdan alınan bilgiler aktarılır. Dosya işlemi olduğu için `try-catch` kullanılır. Yazma adımları şöyledir:

1. Dosya adı `fileName` değişkenine konur.
2. `new FileOutputStream(fileName)` dosyayı oluşturur. Bu nesne `ObjectOutputStream` yapıcısına verilir, böylece iki akış zincirlenir.
3. `writer.writeObject(contactCount)` önce kayıt sayısını yazar.
4. Döngüyle her `Contact` yazılır.
5. Dosya kapatılır.

Hoca kullanıcı girdisinin doğruluğunun, örneğin e-posta biçiminin, ayrıca denetlenmesi gerektiğini, burada bunun yapılmadığını da söyledi.

```
package files;

import java.io.*;
import java.util.Scanner;

public class CreateContacts {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Kac kisi gireceksiniz? ");
        Integer contactCount = Integer.valueOf(input.nextLine());
        Contact[] contacts = new Contact[contactCount];

        for (int i = 0; i < contactCount; i++) {
            System.out.print("Ad: ");
            contacts[i] = new Contact(input.nextLine());
            System.out.print("Telefon: ");
            contacts[i].setPhone(input.nextLine());
            System.out.print("E-posta: ");
            contacts[i].setEMail(input.nextLine());
        }
        input.close();

        String fileName = "contacts.dat";
        try {
            ObjectOutputStream writer =
                new ObjectOutputStream(new FileOutputStream(fileName));
            writer.writeObject(contactCount);
            for (Contact c : contacts)
                writer.writeObject(c);
            writer.close();
            System.out.println("Bilgiler " + fileName + " dosyasina yazildi.");
        } catch (IOException e) {
            System.out.println(e);
        }
```

```
    }
}
```

Hoca çalışma sorusu olarak bunun dizi yerine `ArrayList` ya da `LinkedList` ile yazılmasını önerdi. `ArrayList` için boyut verilebilir, gerekirse boyut artar.

> **Ders sonrası not:** Hoca `ArrayList`'te "yine boyut giriyorsunuz" dedi. Bu zorunlu değildir: `new ArrayList<>()` javac ile denendi, derlendi ve eleman eklendi.

### 6.5 ShowContacts: Aynı Sırayla Okumak

Ters serileştirmede de iki akış zincirlenir: `ObjectInputStream` ve `FileInputStream`. `readObject` metodundan gelen şey varsayılan olarak `Object` türündedir. `Object` bütün sınıfların atası olduğu için kendi sınıfımızda kullanmak istediğimizde tip dönüşümü yaparız. Dersin başındaki konu burada işe yarıyor.

Yazdığımız sıra `Integer`, `Contact`, `Contact`, … olduğu için okuma da aynı sırayla yapılır.

> **Hocanın vurgusu:** Nasıl yazarsanız öyle okumanız gerekir. İlk yazdığımız nesne `Integer` idi, sonrakiler `Contact`. Dosyayı buna uyumlu biçimde okumalıyız.

Hoca bu düzeni PGM resim dosyasına benzetti. PGM dosyası açıldığında başında resmin kaç satır kaç sütun olduğu ve kaç renk içerdiği yazar, ardından veri gelir. Burada da kendimiz bir format belirlemiş olduk: İlk nesne kayıt sayısını söylüyor.

```
package files;

import java.io.*;

public class ShowContacts {
    public static void main(String[] args) {
        try {
            ObjectInputStream reader =
                new ObjectInputStream(new FileInputStream("contacts.dat"));
            Integer contactCount = (Integer) reader.readObject();
            for (int i = 0; i < contactCount; i++) {
                Contact c = (Contact) reader.readObject();
                System.out.println(c);
            }
            reader.close();
        } catch (IOException e) {
            System.out.println(e);
        } catch (ClassNotFoundException e) {
            System.out.println(e);
        }
    }
}
```

**Soru:** `System.out.println(c)` yazınca hangi metot çağrılır? **Cevap:** `toString`. Otomatik çağrılır, çünkü onu tanımlamıştık.

İki kişi girilerek çalıştırıldı, çıktı:

```
A - A - A
B - B - B
```

> **Ders sonrası not:** Hoca `ClassNotFoundException`'ı "dönüşüm olmuyorsa, böyle bir sınıf bulamıyorsa" alınan istisna gibi anlattı. Bu ikisi farklıdır. Çalıştırılarak denendi: Hatalı dönüşüm `ClassCastException` verdi. `ClassNotFoundException` ise dosyadaki sınıfın `.class` dosyası silinince `readObject` sırasında alındı (`java.lang.ClassNotFoundException: V`). `readObject` bu denetimli istisnayı bildirdiği için yakalanması zorunludur. javac ile denendi: `unreported exception ClassNotFoundException`.

### 6.6 Dosyanın İçine Bakmak

Hoca programı çalıştırıp Eclipse proje klasöründe oluşan `contacts.dat` dosyasını metin editöründe açtı. Yalnız dosya adı verildiği için, yani göreli yol kullanıldığı için, dosya programın çalıştığı klasörde oluşmuştu. İçinde `java.lang.Integer`, `java.lang.Number`, `files.Contact` gibi sınıf adları ve girilen metinler görünüyordu.

Hoca burada yazma düzenini Java'nın kendisinin kurduğunu vurguladı. Aynı işi C'de yapsak formatın ayrıntılarına çok daha fazla hâkim olmamız gerekirdi. Hangi alanın önce yazılacağı gibi işleri akış sınıfları halleder.

> **Ders sonrası not:** Hoca dosyada görünen bir "2" karakterinin muhtemelen kayıt sayısı olduğunu söyledi. python ile okundu: İki kayıtlı örnek dosya 195 bayttır, sayım 0'dan başlatılınca `java.lang.Integer` 8. baytta, `files.Contact` 85. baytta başlar. Kayıt sayısı `2` karakteri olarak değil, 77. bayttan itibaren dört bayt `00 00 00 02` olarak yazılıdır; dosyada `2` karakteri hiç geçmez. Metin editöründe bu sayı doğrudan okunmaz.

### 6.7 Yanlış Sırada Okuma Denemesi

Hoca `ShowContacts`'ı değiştirip ilk nesneyi `Integer` yerine `Contact`'a dönüştürdü. Program `ClassCastException` verdi, çünkü okunan şey `Integer` idi. javac ile derlenip çalıştırılarak aynı sonuç alındı: `class java.lang.Integer cannot be cast to class files.Contact`. Sonuç yine aynıdır: Dosya nasıl yazıldıysa öyle okunur.

## 7. Dosya Sonu Sorunu ve Kapsayıcı Nesneler

### 7.1 Kayıt Sayısını Bilmezsek

Kayıt sayısı dosyada yoksa dosya sonuna kadar okumamız gerekir. Hoca, dosyanın sonuna gelindiğini anlamanın güvenli ve etkili bir yolu olmadığını anlattı:

- `hasNext` tarzı bir yöntem kullanılamıyor.
- `available` metodu var ama sorunlu.
- `readObject` dosya sonunda `null` döndürmüyor.

> **Ders sonrası not:** Hoca `reader.hasNext()` için "JDK 8'den itibaren kaldırıldı" dedi. javac ile denendi: JDK 25'te `ObjectInputStream` üzerinde `hasNext` çağrısı `cannot find symbol` verir. Bu metot büyük olasılıkla önceki sürümlerde de hiç yoktu, yani kaldırılmış değildir.

> **Ders sonrası not:** `available` sorunu çalıştırılarak görüldü: İçinde kayıtlar varken `reader.available()` 0 döndürdü, ilk nesne okunduktan sonra da yine 0 döndürdü. Bu yüzden "okunacak nesne var mı" sorusuna cevap vermez.

Çözümlerden biri istisnayı kabullenmektir. Döngünün içine `try` yazılır, `catch` içinde `break` ile döngüden çıkılır. Hocanın sözlü anlattığı bu kalıp şöyle kurulur ve javac ile derlenip çalıştırıldı, kayıtlar okunduktan sonra `EOFException` ile döngü bitti:

```
while (true) {
    try {
        Contact c = (Contact) reader.readObject();
        System.out.println(c);
    } catch (EOFException e) {
        break;
    }
}
```

> **Hocanın vurgusu:** İstisna yakalamanın asıl amacı bu değildir. İstisna yakalama, çalışma sırasındaki hatayı kullanıcıya bildirmek, programı ve okuyucu-yazıcıları sağlıklı biçimde kapatmak içindir. Döngüden çıkmak için kullanmak pek doğru değildir.

### 7.2 Kapsayıcı Nesne: Listeyi Tek Seferde Yazmak

Daha iyi yol kapsayıcı (container) nesne kullanmaktır. Kapsayıcı, bütün verilerin referanslarını tutan bir veri yapısıdır, yani liste ya da harita (map). Döngüyle kayıt yazmak yerine `writer.writeObject(contacts)` deyip listenin kendisini tek seferde yazarız. Okurken de listenin tamamını tek seferde okuruz. Kaç kayıt olduğunu yazmaya da gerek kalmaz.

Hoca bunun veri tabanı işlemlerinde, MVC ve web programlamada çok karşımıza çıktığını söyledi: Kaynakta kaç veri olduğunu bilmediğiniz durumda `LinkedList`, `ArrayList` gibi dinamik yapılar çok işe yarar. Aynı işi C'de yapsak önce `String` ya da satır okuyup parçalara bölmemiz gerekirdi. Java akışları bunu çok kolaylaştırıyor.

Slaytta kalıtımla ilgili bir kural da vardı: `A` ile `B` arasında bir ilişki varsa ikisi de `Serializable` olmalıdır.

> **Ders sonrası not:** Kural kalıtım için bu kadar katı değildir. Çalıştırılarak denendi:
>
> - Üst sınıf `Serializable` ise alt sınıf ayrıca yazmadan serileştirilebilir (alanlar korundu).
> - Yalnız alt sınıf `Serializable` ise üst sınıfın alanı yazılmaz. Okunurken üst sınıfın parametresiz yapıcısı çalıştı, alan yapıcının verdiği değeri aldı. Parametresiz yapıcı yoksa `InvalidClassException: ... no valid constructor` alındı.
> - Üye olarak tutulan nesnenin sınıfı `Serializable` değilse `NotSerializableException` alındı. Bu, kapsayıcıdaki elemanlar için de geçerlidir: `Serializable` olmayan elemanlı `LinkedList` yazılamadı. `ArrayList` ve `HashMap` tek seferde yazılıp okunabildi.

### 7.3 nyp12b: LinkedList ile

Hoca bu sürümü de çalıştırdı. Liste tek seferde yazılır, tek seferde okunur, hiçbir döngü gerekmez. `readObject` sonucu `LinkedList<Contact>` türüne dönüştürülür.

**Soru:** Bu örnekte kullanıcıya "kaç kayıt gireceksiniz" diye sormak gerekli mi? **Cevap:** Hayır. Liste baştan belirlenip kullanıcı kayıt girebilir. Hoca örneğin `-1` girilene kadar dönen bir `while` döngüsü kullanılabileceğini söyledi.

Aşağıdaki kod bu öneriyle kurulmuştur ve javac ile derlenip çalıştırıldı. Çıktı önceki örnekle aynıdır.

```
// CreateContacts.java
package files;

import java.io.*;
import java.util.LinkedList;
import java.util.Scanner;

public class CreateContacts {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        LinkedList<Contact> contacts = new LinkedList<Contact>();

        System.out.print("Ad (bitirmek icin -1): ");
        String name = input.nextLine();
        while (!name.equals("-1")) {
            Contact c = new Contact(name);
            System.out.print("Telefon: ");
            c.setPhone(input.nextLine());
            System.out.print("E-posta: ");
            c.setEMail(input.nextLine());
            contacts.add(c);
            System.out.print("Ad (bitirmek icin -1): ");
            name = input.nextLine();
        }
```

```
        input.close();

        try {
            ObjectOutputStream writer =
                new ObjectOutputStream(new FileOutputStream("contacts.dat"));
            writer.writeObject(contacts);
            writer.close();
        } catch (IOException e) {
            System.out.println(e);
        }
    }
}

// ShowContacts.java
package files;

import java.io.*;
import java.util.LinkedList;

public class ShowContacts {
    @SuppressWarnings("unchecked")
    public static void main(String[] args) {
        try {
            ObjectInputStream reader =
                new ObjectInputStream(new FileInputStream("contacts.dat"));
            LinkedList<Contact> contacts =
                (LinkedList<Contact>) reader.readObject();
            reader.close();
            for (Contact c : contacts)
                System.out.println(c);
        } catch (IOException e) {
            System.out.println(e);
        } catch (ClassNotFoundException e) {
            System.out.println(e);
        }
    }
}
```

**Ders sonrası not:** Hoca `@SuppressWarnings("unchecked")` için "exception'lar için uyarıları bastır" dedi. Bu annotation'ın istisnalarla ilgisi yoktur. Jenerik tipe yapılan denetlenmemiş dönüşüm uyarısını gizler. javac ile denendi: `-Xlint:unchecked` ile `(Contact) reader.readObject()` uyarı vermedi, `(LinkedList<Contact>) reader.readObject()` ise `unchecked cast` uyarısı verdi. `serialVersionUID` uyarısını gizleyen ise `@SuppressWarnings("serial")`'dır, bu da denendi.

### 7.4 nyp12c: Dizi ile

Dizi de tek seferde yazılıp okunabilir. Okurken `Contact` dizisine dönüştürülür: `Contact[] contacts = (Contact[]) reader.readObject();`. Çalıştırılarak denendi, iki elemanlı dizi aynen geri geldi.

### 7.5 Diğer Okuma Sınıfları

Hoca Java'nın kendi eğitim sayfasını açarak `java.io` altında bit, bayt ve satır okuyan birçok sınıfın bir hiyerarşi içinde bulunduğunu, C'dekine benzer tampon (buffer) mekanizmaları olduğunu, tek satır okuma gibi işlemlerin yapılabildiğini gösterdi. Ancak nesne yazılmış bir dosyadan tek satır okursanız nesnenin düzeni bozulur. Yine nasıl yazıldıysa öyle okumak en doğrusudur.

## 8. İç Sınıflar

Hoca dosya konusunu kapattıktan sonra kalan konuları sıraladı: thread'ler, iç sınıflar, birim testi. İç sınıfa geçmeden önce veri yapıları dersinde kullandığı, Sedgewick'e ait bir slaytı gösterdi.

### 8.1 Motivasyon: Bağlı Listenin Düğümü

Bir yığın (stack) ya da kuyruk (queue) sınıfını bağlı listeyle yazmak istiyoruz. Bağlı liste düğümlerden oluşur. Java'da liste bir sınıf, her düğüm de ayrı bir sınıf olacak. Düğüm sınıfını başka bir yerde kullanmayacaksak, yani yalnızca bağlı listeye özelse, ayrı bir dosya açıp tanımlamaya gerek yoktur. İç sınıf (inner class) tanımlarız.

İç sınıf, bir sınıfın içinde tanımlanan sınıftır. Kendi bloğu vardır. Düğümün içinde bir `item` ve kendi türünden bir `next` bulunur. İki ayrı `.java` dosyası yerine tek dosyada, dış sınıfın altında `private` bir sınıf yazmış oluruz. `private` yapmamızın nedeni dışarıdan erişilmesini engellemektir.

Aşağıdaki kod hocanın anlatımından kuruldu, javac ile derlenip çalıştırıldı. Çıktısı `b`, `a` oldu, yani son eklenen ilk çıktı.

```
public class LinkedStack<Item> {
    private Node first;

    private class Node {
        Item item;
        Node next;
    }

    public void push(Item item) {
        Node oldFirst = first;
        first = new Node();
        first.item = item;
        first.next = oldFirst;
    }

    public Item pop() {
        Item item = first.item;
        first = first.next;
        return item;
    }

    public static void main(String[] args) {
        LinkedStack<String> s = new LinkedStack<String>();
        s.push("a");
        s.push("b");
        System.out.println(s.pop());
        System.out.println(s.pop());
    }
}
```

### 8.2 İç Sınıfın Özellikleri

- Dış sınıfın `private` değişkenleri dahil bütün üyelerine erişebilir.
- `private` tanımlanırsa aynı paketteki diğer sınıflardan gizlenir.
- GUI'de ve çok iş parçacıklı programlamada çok kullanılır. Özellikle anonim, yani isimsiz sınıf olarak: "Butona tıklanınca şu işi yap" gibi.
- Slayta göre içinde statik metot tanımlanamaz.

  **Ders sonrası not:** Son madde eski sürümler için geçerlidir. javac ile denendi: `class X { class In { static void s(){} } }` satırı `-source 15` ile `Illegal static`

`declaration in inner class` hatası verdi, `-source 16` ve JDK 25 varsayılanıyla derlendi.

Hocaya göre iç sınıf kalıtıma biraz benzer ama kalıtım değildir. Nesneye yönelik bir kavramdan çok programlamayla ilgili bir kavramdır.

### 8.3 Örnek 1: private Employee

`Person` sınıfı ad bilgisini tutar. İçinde, "bazı kişiler çalışandır" demek için `Employee` iç sınıfı tanımlanmıştır. Bu sınıfın maaş bilgisi, yapıcısı ve kendi metotları vardır. `main` metodu aynı sınıfta olduğu için `Employee` burada kullanılabilir. Dizinin bir elemanına 7500 maaşlı bir çalışan konur, sonra kayıtlar ekrana yazdırılır.

```
public class Person {
    private String name;

    public Person(String name) { this.name = name; }

    private class Employee {
        private double salary;

        public Employee(double salary) { this.salary = salary; }

        public String toString() {
            return name + " - " + salary;
        }
    }

    public static void main(String[] args) {
        Person person = new Person("Kisi");
        Employee staff = person.new Employee(7500);
        System.out.println(staff);
    }
}
```

javac ile derlenip çalıştırıldı, çıktı: `Kisi - 7500.0`. Burada `name`, dış sınıfın `private` alanıdır ve iç sınıf ona doğrudan erişir.

> **Ders sonrası not:** Hoca nesnenin `new Employee` ile üretildiğini söyledi. Statik olmayan iç sınıf, statik `main` içinden doğrudan `new Employee(...)` ile üretilemez. javac ile denendi: `non-static variable this cannot be referenced from a static context`. Yukarıdaki gibi `person.new Employee(...)` yazmak gerekir. Slayttaki yazım altyazıdan okunamıyor.

`main` metodu başka bir sınıfa taşınırsa hata alınır. Hoca bunun "Employee bulunamadı" hatası olacağını söyledi. javac ile denendi: Kısa ad için `cannot find symbol`, `Person.Employee` yazılırsa `Person.Employee has private access in Person` hatası alındı.

### 8.4 Örnek 2: public Employee ve import

Aynı sınıfta `Employee` bu kez `public` yapılır. Başka bir sınıftan kullanılabilir, ama `import` gerekir. Çünkü o sınıf `Person`'ı görür, `Employee`'yi göremez. javac ile denendi: `import` satırı kaldırılınca aynı pakette olmasına rağmen `cannot find symbol` hatası alındı.

```
// Person.java
package nyp13b;

public class Person {
    private String name;

    public Person(String name) { this.name = name; }

    public class Employee {
```

```
        private double salary;

        public Employee(double salary) { this.salary = salary; }

        public String toString() {
            return name + " - " + salary;
        }
    }
}

// InnerClassDirect.java
package nyp13b;

import nyp13b.Person.Employee;

public class InnerClassDirect {
    public static void main(String[] args) {
        Person person = new Person("Kisi");
        Employee staff = person.new Employee(7500);
        System.out.println(staff);
    }
}
```

### 8.5 Örnek 3: Dış Sınıftan Açık Kapı (enlist)

`private` iç sınıfa diğer sınıflardan erişmek için dış sınıfta `public` bir metot kullanılır. `Person` sınıfı `Employee` türünden bir referans alan tutar. `enlist` metodu bir `Employee` üretir ve bu referansa bağlar. `toString` ise `employee` alanı `null` değilse maaşı da yazar. Metot çağrılmadıysa alan varsayılan olarak `null` kalır. Yani `null` olmaması, "bu kişi aynı zamanda çalışan" demektir.

```
// Person.java
public class Person {
    private String name;
    private Employee employee;

    public Person(String name) { this.name = name; }

    private class Employee {
        private double salary;

        public Employee(double salary) { this.salary = salary; }

        public double getSalary() { return salary; }
    }

    public void enlist(double salary) {
        employee = new Employee(salary);
    }

    public String toString() {
        if (employee != null)
            return name + " - " + employee.getSalary();
        return name;
    }

}

// Test.java
public class Test {
    public static void main(String[] args) {
```

```
        Person p1 = new Person("Kisi 1");
        Person p2 = new Person("Kisi 2");
        p2.enlist(1000);
        System.out.println(p1);
        System.out.println(p2);
    }
}
```

javac ile derlenip çalıştırıldı, çıktı:

```
Kisi 1
Kisi 2 - 1000.0
```

> **Ders sonrası not:** Hoca bu örnekte "işte bu polymorphism" dedi. Anlatılan yapıda kalıtım ve geçersiz kılma yoktur, `enlist` sıradan bir metot çağrısıdır. Bu büyük olasılıkla bir söz sürçmesidir.

### 8.6 Dış Sınıf Nesnesine Ulaşmak

İç sınıftan dış sınıf nesnesinin referansına `DisSinifAdi.this` ile ulaşılır, örneğin `Person.this`. Hoca buna çok ihtiyaç duyulmadığını söyledi. javac ile denendi: İç sınıfta aynı adlı bir `x = 4` alanı ve dışta `x = 3` varken `x + Refl.this.x` sonucu 7 oldu.

## 9. Görsel Programlamaya Giriş: C# ile Pencere ve Olaylar

Hoca thread konusuna bugün girmeyeceklerini söyledi ve pencere programlamaya geçti. Mantık C#'ta çok kolay olduğu için önce C#'tan gösterdi, Java karşılığının sonra anlatılacağını belirtti.

### 9.1 İki Yol: Sürükle-Bırak ve Kod

Masaüstünde ya da webde görsel programlama yaparken iki yol vardır: Butonlar gibi grafik öğeleri kendiniz kodlarsınız ya da tasarımcı (builder, designer) araçlardan yararlanırsınız. Bu araçlar NetBeans'te, Eclipse'te eklenti olarak, Visual Studio'da vardır. Açılan pencerenin adı C#'ta `Form`, Java'da `JFrame`'dir.

Hoca önce Visual Studio'da Windows Forms projesi açtı. Araç kutusundan buton ve metin kutusu sürükledi. Butona tıklanınca `textBox1.Text = "Merhaba"` olmasını sağladı.

Hoca bunun aslında sınıflar, kalıtım ve ilişkilendirmenin (association) bir uygulaması olduğunu söyledi.

### 9.2 GUI Nesnesinin Dört Parçası

**Soru:** Bir UML şemasında kaç farklı alan vardı? **Cevap:** Üç: en tepede sınıfın adı, altında özellikler, onun altında metotlar.

GUI'de ise dört parça vardır, çünkü ekranda gördüğümüz her şey bir nesnedir:

- **Ad:** Nesnenin adı.
- **Özellikler (property):** Rengi, üzerindeki yazı gibi görünen özellikler.
- **Metotlar:** Örneğin butonu sağa ya da sola kaydırmak.
- **Olaylar (event):** Butona tıklamak, metin kutusuna yazmak, fareyi hareket ettirmek.

Hocanın olay örneği kitap aramaydı. Arama kutusuna `A`'ya basınca A ile başlayanlar, `B`'ye basınca AB ile başlayanlar gelir, `B`'yi silince yine A ile başlayanlara dönülür. Arkada sürekli bir sorgu çalışır. Her tuşa basış, fareyi hareket ettirmek, hep birer olaydır. Bu olaylara bağlanan mekanizma olay işlemedir (event handling).

**Soru:** .NET arayüzü başka hangi dilleri destekliyor? **Cevap:** Burada yazdığımız kod yalnız C# içindir. C++ için de bir karşılığı olabilir, ama onun için ayrı proje oluşturmak gerekir.

### 9.3 Kodla Pencere: Kalıtım

Hoca tasarımcı dosyasını sildi. Geriye `Program.cs` kaldı. Hoca bunun, Java'da yazdığımız konsol programıyla aynı olduğunu söyledi. `main` metodu buradadır, program buradan başlar ve bir form nesnesi üretir.

Sonra `Pencere` adında normal bir sınıf oluşturdu. Bu sınıf `Form`'dan kalıtım alınca görsel pencere elde edilir. Bunun için `Form`'un bulunduğu ad alanı `using` ile eklenir, bu Java'daki `import`'un karşılığıdır.

Pencerenin boyutları için bir yapıcı yazıldı: `public Pencere(int w, int h)`. `Program.cs` tarafında `Pencere` nesnesi 300'e 600 olarak üretilir. İlk denemede değerler yalnız gönderildiği, hiçbir yere aktarılmadığı için boyutun değişmediği görüldü. `Form` sınıfının tanımına gidilince `Width`, `Height`, `Text` gibi özellikler görünür. Kalıtım aldığımız için bunlara doğrudan erişebiliriz: `this.Width = w`, `this.Height = h`, `this.Text = "Deneme"`. Hoca `Text`'in, başlık çubuğundaki yazıyı değiştirdiğini gösterdi. Çalıştırınca 300'e 600'lük ve başlığı "Deneme" olan pencere geldi.

Hoca geliştirme ortamındaki simgeleri de gösterdi: Anahtar işareti özellik, şimşek işareti olay, diğerleri metottur.

### 9.4 Fare Olayları: MouseMove ve MouseClick

`this.Mouse` yazınca fareyle ilgili olaylar listelenir: tıklama, çift tıklama, bırakma, tekerlek gibi. `MouseMove` farenin her hareketinde oluşur, hareket ettikçe defalarca çalışır.

> **Ders sonrası not:** Hoca `MouseHover`'ı da hareket olayı gibi anlattı. Hareket olayı `MouseMove`'dur, `MouseHover` ise büyük olasılıkla imleç nesne üzerinde kısa süre durunca oluşur.

Hedef: Fare her hareket ettiğinde koordinatları pencerenin başlığına yazmak. Bunun için olay ile metot arasında bağlantı kurulur. Hoca bunu istisna yakalamaya benzetti: Orada "istisna olunca şunları yap" demiştik, burada "olay olunca şunları yap" diyoruz. C#'ta bu bağlantının yolu `+=` operatörüdür. Geliştirme ortamı metot adını önerir, Tab'a basınca metodu da oluşturur.

Metodun iki parametresi vardır:

- `object sender`: Olayı kim oluşturdu? Fare, klavye ya da başka kontrol olabilir, bu yüzden türü `object`'tir.
- `MouseEventArgs e`: Olayın bilgileri. `e.X`, `e.Y` koordinatları, ayrıca tıklama bilgileri.

Çalıştırınca fare hareket ettikçe başlık değişti, ama hareketler yalnız pencerenin beyaz alanı içinde yakalandı. Sabit bir yerde tıklayınca hiçbir şey olmadı, çünkü tıklama başka bir olaydır. Bu yüzden `MouseClick` da bağlandı ve `MessageBox` ile bir açılır pencere gösterildi.

### 9.5 Buton ve Controls

Kodla buton eklemek için önce referans sınıfın üyesi olarak yazılır, yapıcıda `new Button()` ile nesne oluşturulur. Boş yapıcısı vardır. Sonra `Width = 50`, `Height = 60`, `Text = "Tıkla"` verilir.

Hoca çalıştırdığında buton görünmedi. Çünkü buton nesne olarak var ama formun kontrollerine eklenmemiş. Formda bu nesneleri tutan bir `Controls` yapısı vardır, `this.Controls.Add(b1)` ile eklenince buton göründü.

Konum için `Location` özelliği kullanılır. Bu özellik `Point` türündedir, yani önce nesne üretilir: `new Point(50, 50)`.

> **Ders sonrası not:** Hoca `using System.Drawing.Point` yazınca derleyicinin kabul etmemesine anlam veremedi. C# derleyicisiyle (csc) denendi: `error CS0138: A 'using namespace' directive can only be applied to namespaces.` `using` ile tür değil ad alanı eklenir, yani `using System.Drawing;` yazılmalıdır. Aşağıdaki kodda böyle yazıldı.

### 9.6 Metin Kutusu ve Konum Hesabı

Metin kutusu için de aynı adımlar izlenir. Hoca kutunun butonun tam altına gelmesini istedi ve bunu sayılarla gösterdi:

- **Verilen:** Butonun sol üst köşesi $(50, 50)$, yüksekliği 60.
- **İşlem:** Butonun alt kenarı $y = 50 + 60 = 110$ (python ile hesaplandı).
- **Sonuç:** Hoca kutuyu x ekseninde yine 50'den, y ekseninde 120'den başlattı. Böylece kutu butonun $120 - 110 = 10$ birim altında kalır (ara adım eklendi).

Kutunun yüksekliği 50 verildi ve `Controls.Add(t1)` ile forma eklendi. Hoca sıranın önemli olmadığını gösterdi: Önce buton sonra kutu ya da tersi eklenebilir.

### 9.7 Butonun Click Olayı

Çalıştırınca formun üzerine tıklamak mesaj kutusunu açıyordu ama butona tıklamak hiçbir şey yapmıyordu. Çünkü her nesnenin olayları kendine özgüdür. Butonun `Click` olayına `+=` ile bir metot bağlanır ve metin kutusuna "Merhaba" yazılır.

Bütün sınıf aşağıdadır. Windows Forms bu makinede yok. Kod bu yüzden gerçek pencereyle çalıştırılmadı. Yalnız sözdizimi, gerçek sınıflar yerine yazılan basit taklit sınıflarla C# derleyicisinden (csc) geçirildi.

```
using System;
using System.Drawing;
using System.Windows.Forms;

class Pencere : Form
{
    Button b1;
    TextBox t1;

    public Pencere(int w, int h)
    {
        this.Width = w;
        this.Height = h;
        this.Text = "Deneme";

        this.MouseMove += Pencere_MouseMove;
        this.MouseClick += Pencere_MouseClick;

        b1 = new Button();
        b1.Location = new Point(50, 50);
        b1.Width = 50;
        b1.Height = 60;
        b1.Text = "Tıkla";
        this.Controls.Add(b1);
        b1.Click += B1_Click;

        t1 = new TextBox();
        t1.Location = new Point(50, 120);
        t1.Height = 50;
        this.Controls.Add(t1);
    }

    private void Pencere_MouseMove(object sender, MouseEventArgs e)
    {
        this.Text = "x: " + e.X + " -- y: " + e.Y;
    }

    private void Pencere_MouseClick(object sender, MouseEventArgs e)
    {
```

```
        MessageBox.Show("Mouse tıklandı");
    }

    private void B1_Click(object sender, EventArgs e)
    {
        t1.Text = "Merhaba";
    }
}

// Program.cs içinde
Application.Run(new Pencere(300, 600));
```

### 9.8 Bağlama Mekanizması

Hoca, burada anlattıkları içinde yeni olan tek şeyin olay mekanizması olduğunu vurguladı. Geri kalanı kalıtım ve üye değişkenlere erişimdir. Bağlama farklı dillerde şöyledir:

- **C#:** += operatörü.
- **Java:** `add...` metodu.
- **Python:** `connect` metodu. Fare hareketini bir metoda bağlarsınız.

Hoca bunun, yapısal programlamada gösterilen fonksiyon işaretçilerine (function pointer) benzediğini söyledi. C#'taki karşılığının `delegate` yapısı olduğunu ekledi: Olay gerçekleşince bağlanan metot çağrılır.

### 9.9 Neden Kodla Yazıyoruz?

Uygulamanız basitse, örneğin 10 buton ve 5 metin kutusu varsa sürükle-bırak kolaydır. Ama bir otobüs firmasının koltuklarını ya da bir stadyumu tasarlıyorsanız 40 bin butonu sürükle-bırak yapmak mümkün değildir, yönetilemez.

> **Hocanın vurgusu:** Bu tür arayüzlerin programlanması lazım. Butonlar dizide (`Button[] b2`) ya da listede (`List<Button> L`) saklanıp istenildiği kadar eklenir.

Hoca, otobüs ya da sinema sitelerinde koltuk seçtiğiniz ekranın arkasında da programla oluşturulmuş bir arayüz olduğunu, bazı ortamlarda bu arayüzün Android'deki gibi XML ya da web-deki gibi HTML biçiminde yazıldığını söyledi.

Buradaki örnekte iş mantığı, veri tabanı ya da bilimsel hesap yoktur. Yalnız kullanıcı arayüzü tasarımı ve nesneye yönelik konuların basit bir uygulaması vardır. Hoca bunun aynı zamanda dönemin genel bir tekrarı olduğunu söyledi.

## 10. Ders Sonu Sorusu: İç Sınıfların Bellek Maliyeti ve Java'da Olay Bağlama

### 10.1 İç Sınıf Belleği Yorar mı?

**Soru:** İç içe sınıf yazmak mantık ve adlandırma açısından kolaylık sağlıyor. Ama iç sınıfın dış sınıfa referans tuttuğu için RAM'i yorduğuna dair yorumlar var. Bu doğru mu?

**Cevap:** Hoca, MVC gibi çok sınıflı web programlarında ve diğer uygulamalarda iç sınıf kullanmanın çok mantıklı olmadığını söyledi. Bir `Node` sınıfı oluşturup ondan nesne üretmek daha mantıklıdır, yoksa başka yerde yeniden gerektiğinde tekrar yazılır. İç sınıf fazladan referans ve üye değişken sakladığı için belleği fazladan kullanır. Anonim sınıfta da başlı başına bir nesne oluşturulur. Ama amaç başkası tarafından kullanılmayacak, görülmesi engellenecek basit bir sınıf yazmaksa kodlama ve belki güvenlik açısından faydalıdır. Yani bellek açısından kötü bir tarafı vardır.

> **Ders sonrası not:** Gizli referansın varlığı sınıfa bağlıdır. JDK 25'te javac ile derleyip alanları yansıma (reflection) ile listeleyerek denendi:
>
> - Dış sınıfın alanını kullanan iç sınıfta `this$0` alanı var.
> - Dış sınıfı hiç kullanmayan iç sınıfta `this$0` yok.
> - `Serializable` olan iç sınıfta, dış sınıfı kullanmasa da `this$0` var.

- `static` iç içe sınıfta `this$0` yok.

Yani yorumlar her durumda doğru değildir. Dış nesneye ihtiyaç duymayan bir sınıf `static` yazılırsa bu referans baştan olmaz.

### 10.2 Java'da Buton: ActionListener

Hoca, gelecek derste anlatacağı konuyu bu soru üzerine kısaca açtı. Bir eğitim sayfasından Java'da olay bağlama yöntemlerini gösterdi. Java'da pencere `JFrame`, buton `JButton`'dır. Bizim `+=` karşılığımız `button.addActionListener(...)`'dır. `ActionListener` bir arayüzdür ve `actionPerformed` metodunun yazılmasını zorunlu kılar.

Hocanın anlattığı evrim şöyledir:

1. **Her olay için ayrı sınıf:** Buton tıklaması için bir sınıf, form tıklaması için bir sınıf. Bir sürü sınıf ortaya çıkar, hoca bunu çok kötü buluyor.
2. **Anonim iç sınıf:** Ayrı bir yere gitmeden `new ActionListener() { ... }` yazılır. Hocaya göre göze hoş görünmeyen bir koddur.
3. **Lambda ifadeleri:** Bu yöntem çıkınca önceki yazımlar eskidi. Hoca bu yüzden iç sınıfların da biraz geride kaldığını söyledi.

Hoca bu yapının Swift ve Kotlin'deki closure yapısına çok benzediğini de söyledi. Python'da PyQt örneğini gösterdi: Buton oluşturulur ve `clicked.connect` ile `say_hello` fonksiyonuna bağlanır. Ona göre bu en güzel çözümlerden biridir.

İlk yöntem, sayfadaki örneğe benzer biçimde kuruldu ve javac ile derlendi. Görüntü olmayan modda `JFrame` açılamadığı için (`HeadlessException`) pencere çalıştırılmadı.

```
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.JButton;
import javax.swing.JFrame;

public class ActionEventDemo implements ActionListener {
    JFrame frame = new JFrame();
    JButton button = new JButton("Tikla");

    ActionEventDemo() {
        button.setBounds(50, 50, 50, 60);
        button.addActionListener(this);
        frame.add(button);
        frame.setLayout(null);
        frame.setSize(300, 600);
        frame.setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {
        System.out.println("Butona tiklandi");
    }

    public static void main(String[] args) {
        new ActionEventDemo();
    }
}
```

İkinci yöntem aşağıdadır. `JButton` üzerinde `doClick()` ile tıklama benzetilerek çalıştırıldı, metot çağrıldı.

```
button.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        System.out.println("Anonim sinif: tiklandi");
    }
});
```

**Ders sonrası not:** Hoca soru üzerine bu yapıyı “delegate” diye nitelendirdi. Java’da anonim sınıfla yapılan iş, `ActionListener` arayüzünü uygulayan bir nesne oluşturup butona vermektir. C#’taki `delegate` ile amaç benzer, ama büyük olasılıkla Java’da aynı adda bir dil yapısı yoktur.

**Ek bilgi (derste anlatılmadı):** Hocanın sözünü edip kodunu göstermediği lambda yazımı şöyledir: `button.addActionListener(e -> System.out.println("Lambda: tiklandi"));`. `ActionListener` tek metotlu bir arayüz olduğu için derleyici bu ifadeyi `actionPerformed` metodu olarak kabul eder. javac ile derlendi ve `doClick()` ile çalıştırıldı, metot çağrıldı.

## Dersin Toparlaması

Bu derste önce kalıtım hiyerarşisinde nesneler arası tip dönüşümünü gördük: Yukarı dönüşüm otomatik ve güvenlidir, aşağı dönüşüm elle yapılır ve `instanceof` ile kontrol gerektirir. Dosyalarda ana fikir “nasıl yazıldıysa öyle okunur” oldu. `File` sınıfıyla dosya yönetimini, `ObjectOutputStream` ve `ObjectInputStream` zinciriyle de nesneleri diske yazıp okumayı öğrendik. Okunan nesnenin `Object` olarak gelmesi tip dönüşümünü yeniden gerekli kıldı. Kayıtları tek tek yazmak yerine kapsayıcı nesneyi tek seferde yazmak daha temiz bir çözüm oldu.

Sonra iç sınıfları `Node` ve `Employee` örnekleriyle gördük. C# ile pencere, buton ve metin kutusu oluşturarak olay işleme mekanizmasını tanıdık ve kalıtımın burada nasıl kullanıldığını gördük. Hoca kalan konular olarak thread’leri ve birim testini saydı, Java’da GUI için anonim sınıf ve `ActionListener` yapısını sonraki derste göstereceğini söyledi.

**Not:** İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.